

Assignment 3—Reinforcement Learning

Name : Tong-Hou Cheong (張東浩)

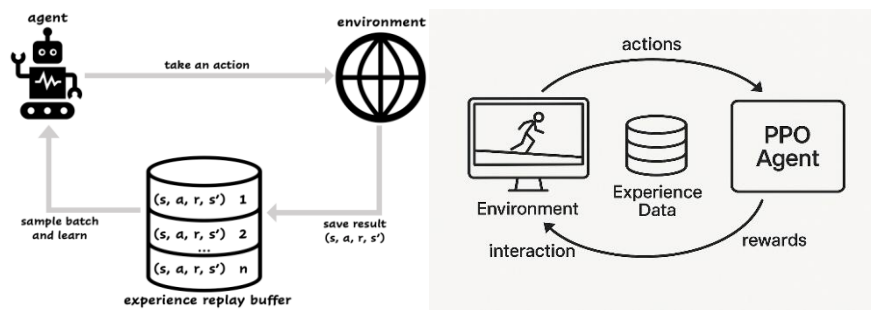
Student ID : 413551038

GitHub : <https://github.com/doctor-66/535505-Artificial-Intelligence--assignment3>

1.Introduction

1.1 Research Question

In Assignment 3, I investigate how different reinforcement learning design choices affect agent performance across environments with different levels of complexity. In particular, the main research question is how algorithm design and hyperparameter settings influence learning stability, sample efficiency, and final performance. Another focus of this assignment is to compare how **value-based learning (Deep Q-learning Network, DQN)** and **policy-based learning (Proximal Policy Optimization, PPO)** behave under different environment.



(Illustration of DQN agent (left half) and PPO agent(right half)).

1.2 Motivation

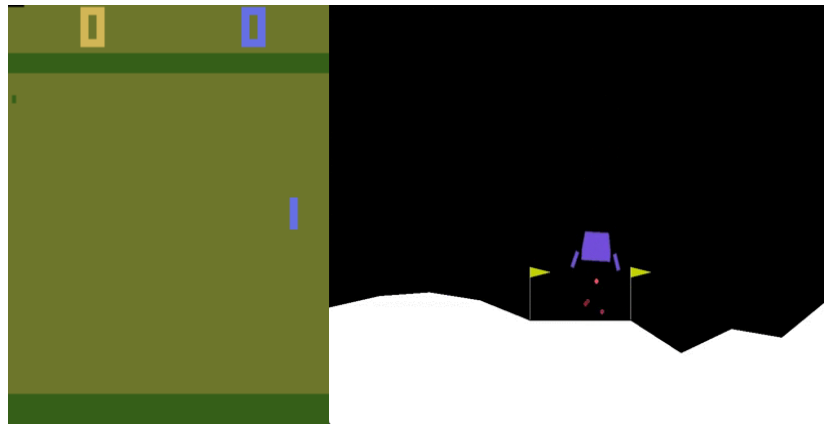
Reinforcement learning performance often depends not only on the choice of algorithm, but also on how the algorithm is implemented and tuned for a specific environment. Small changes in exploration strategy, replay settings, or update frequency can strongly affect convergence speed and stability. The **Pong** environment is a standard Atari benchmark that requires visual perception, temporal information, and long-horizon decision making. In contrast, **LunarLander-v3** is a lower-dimensional control task that allows more controlled analysis of training behavior and makes it easier to observe how hyperparameter choices affect learning.

DQN was selected for the Atari task because it is one of the most classical and influential value-based deep reinforcement learning methods. It extends Q-learning with a neural network and introduces two important mechanisms, experience replay and a target network (Mnih et al., 2015), which improve training stability in high-dimensional settings.

PPO was selected for the control task because it is a widely used policy optimization method that is typically more stable in continuous training and less sensitive to value overestimation issues, making it suitable for analyzing training dynamics in a structured state-space environment (Schulman et al., 2017).

1.3 Tasks Overview

This assignment uses 2 RL strategies: DQN for the Atari task (**Pong**) and PPO for the control task (**LunarLander-v3**), in order to compare value-based and policy-based learning under different environment characteristics. Since **Pong** uses image-based observations, it provides a suitable setting for testing DQN in a high-dimensional visual decision-making problem. The second task is **LunarLander-v3**, which serves as a lower-dimensional control task. Unlike **Pong**, **LunarLander-v3** provides structured state features instead of raw image input, making it useful for comparing how the same value-based reinforcement learning framework behaves under a simpler observation space. Together, these two tasks allow the project to examine DQN from both a visual Atari setting and a lower-dimensional control setting.



(Pong (left half) and Lunarlander (right half) game frame.)

2. Method

2.1 DQN and PPO Overview

DQN is a value-based reinforcement learning method that learns an action-value function $Q(s,a)$ to estimate the expected long-term return of taking action a in state s . Unlike classical tabular Q-learning, DQN uses a neural network to approximate $Q(s,a)$, which makes it applicable to large or high-dimensional state spaces. To improve training stability, DQN typically uses experience replay (sampling past transitions from a replay buffer) and a target network (a slowly updated copy of the Q-network) to reduce correlation in training data and stabilize temporal-difference targets.

PPO is a policy-gradient method that directly optimizes a stochastic policy $\pi(a | s)$ to maximize expected return. PPO is commonly implemented with an actor-critic

structure, where the actor outputs an action distribution and the critic estimates the state value $V(s)$ for variance reduction. Compared with standard policy-gradient methods, PPO constrains policy updates using a clipped surrogate objective, preventing overly large policy changes in a single update and improving training stability.

2.2 Task 1: Pong with DQN

Since **Pong** provides raw image observations, additional preprocessing was required before training. The RGB game frames were converted to grayscale, resized to 84×84 , and stacked over 4 consecutive frames to form the input state. This design allows the RL-agent to capture both spatial information and short-term motion information from the game. A CNN-based DQN was used for Pong so that the model could extract visual features directly from the stacked image frames. The experiments mainly focused on how DQN training behavior changes under different settings, including exploration-related hyperparameters and update-related design choices.

2.3 Task 2: LunarLander with PPO

LunarLander-v3 provides a more controlled setting for analyzing training behavior. Unlike Pong, **LunarLander-v3** uses a structured state vector rather than image observations, so no image preprocessing or frame stacking was needed. For this task, PPO was used as the learning algorithm, which is a policy-gradient method that optimizes the policy directly and typically provides more stable updates through its clipped objective. In this assignment, **LunarLander** experiments were designed to study how PPO training behavior changes under different hyperparameter settings, and to compare its learning dynamics with DQN on the Atari task.

2.4 Evaluation

For both tasks, agent performance was evaluated periodically during training using the **average reward** obtained from evaluation episodes. The reported results include **mean evaluation reward**, **best evaluation reward**, and **final evaluation reward**. In addition, the number of episodes or environment steps required to reach a predefined reward threshold was recorded when applicable, in order to reflect sample efficiency. For the **Pong** task trained with DQN, evaluation was reported as a function of environment steps and episodes, which provides a clear view of learning stability and convergence behavior in the Atari setting. For the **LunarLander** task trained with PPO, evaluation was performed at fixed intervals during policy updates and summarized in the same reward-based metrics. Overall, this evaluation protocol allows comparisons not only in terms of final performance, but also in learning efficiency and training stability across different algorithms and environments.

3. Experimental setup

Main experiment settings table

Task	Environment	Algorithm	Input	Network	Hyperparameters
Task 1	Pong	DQN	4 stacked grayscale frames	CNN	lr, batch size, replay size, target update freq, epsilon setting
Task 2	LunarLander-v3	PPO	state vector	MLP	lr, gamma, GAE lambda, clip_eps, entropy_coef, rollout_steps, update_epochs, minibatch_size, eval_every

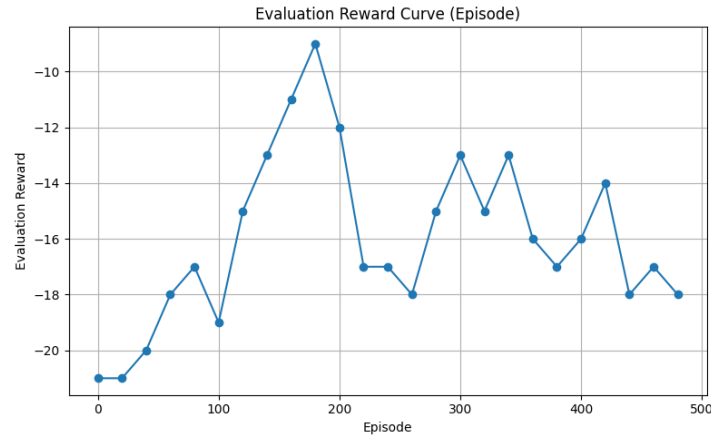
4. Experiments

4.1 Task 1: Pong baseline

For the **Pong** baseline, the agent was trained with a CNN-based Q-network using stacked image frames as input. The main hyperparameters are summarized (**batch size** 32, **replay buffer size** 100,000, **learning rate** 1×10^{-4} , **discount factor** γ 0.99, **ϵ -greedy** exploration with $\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{decay}} = 0.999995$, $\epsilon_{\text{min}} = 0.1$, target network update frequency 1000, and replay start size 10,000; 500 **episodes** in total). As shown in **Figure 1**, the evaluation reward improved from the early stage (around -21) and reached a best value of -9 , indicating the agent began to move away from purely random behavior. However, the overall performance remained limited and unstable under the baseline setting, with the final evaluation reward returning to -18 , which is consistent with the noisy and slow learning dynamics commonly observed in visually complex Atari tasks. **Table 1** reports the baseline performance summary of the agent, the agent achieved a **mean evaluation reward of -16.0** , a **best evaluation reward of -9.0** , and a **final evaluation reward of -18.0** , while the predefined threshold was **not reached**. The total training scale was **3,074,267 environment steps** and **3,064,268 updates**.

Table 1. Pong environment baseline model performance

Mean Eval Reward	-16.0
Best Eval Reward	-9.0
Final Eval Reward	-18.0
Threshold Episode	Not reached
Threshold Env Steps	Not reached
Total Env Steps	3074267
Total Updates	3064268



(Figure 1. Baseline model of reward evaluate every 20 episodes.)

4.2 Task 1: Pong environment ablation studies

Table 2. Pong environment ablation variables

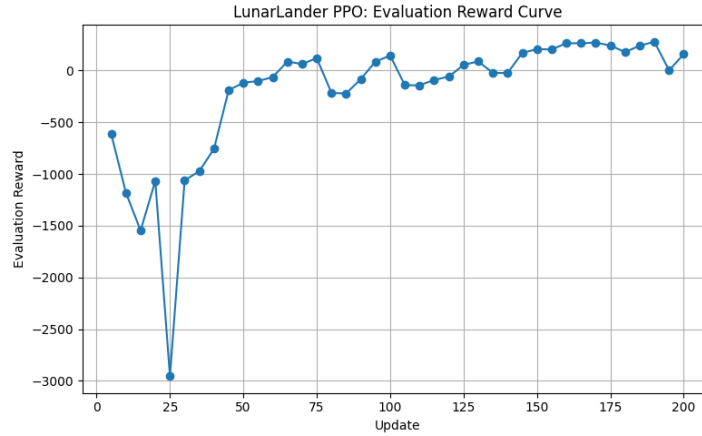
Changed variable	Values
Epsilon_decay	0.99999 / 0.999999
Target_update_frequency	500 /5000
Replay_start_size	5000 / 20000
Train_per_step	2 / 5

4.3 Task 2: LunarLander baseline

For the **LunarLander PPO baseline**, the main hyperparameters are summarized (**learning rate** 3×10^{-4} , **discount factor** γ 0.99, **GAE** λ 0.95, **clipping parameter** ϵ 0.2, **entropy coefficient** 0.01, **value loss coefficient** 0.5, **rollout steps** 2048, **update epochs** 10, **batch size** 64, and evaluation performed every 5 updates; 200 **updates** in total). As shown in **Figure 2**, the evaluation reward increased substantially over training updates, indicating that PPO learned an effective control policy for LunarLander under the baseline setting. **Table 2** reports the baseline performance summary of the LunarLander PPO agent. The mean, best, and final evaluation rewards summarize overall performance across evaluations, while **threshold_update** indicates the first update at which the agent reaches a predefined evaluation reward threshold (if reached). In addition, **total_env_steps** summarizes the overall amount of environment interaction during training. In this baseline run, the agent achieved a **best evaluation reward of 278.33** and a **final evaluation reward of 158.19**, with **threshold_update** = 150 and **total_env_steps** = 409600. The relatively low mean evaluation reward (−213.09) is mainly affected by the poor performance in early evaluations before the policy started to improve.

Table 2. LunarLander environment baseline model performance

Mean Eval Reward	-213.09
Best Eval Reward	278.33
Final Eval Reward	158.19
Threshold Update	150
Total Env Steps	409600



(Figure 2. Evaluation reward curve of the LunarLander baseline every 5 updates.)

4.4 Task 2: LunarLander environment ablation studies

Table 3. LunarLander environment ablation variables

Changed variable	Values
Epsilon_Clip	0.1 / 0.3
Entropy_coefficient	0 / 0.3 / 0.5
Learning Rate	1×10^{-3} / 1×10^{-4}

5. Results

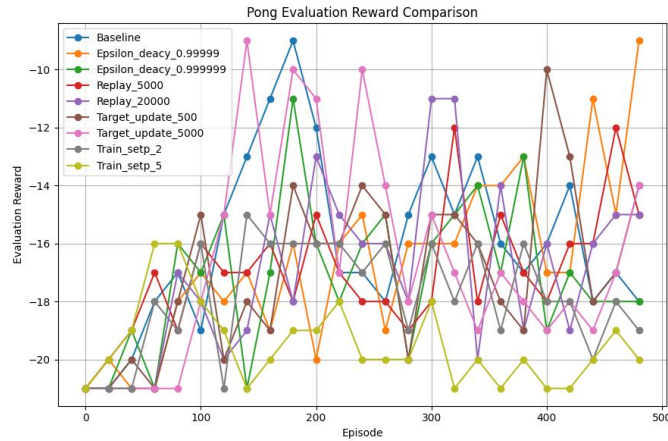
5.1 Pong results

From **Table 3** and **Figure 3**, the **Pong** environment ablation results show that training remained **noisy and unstable across settings**, with evaluation rewards fluctuating mostly between **-20 and -9** over ~2–3M environment steps. The **baseline** achieved a mean eval reward of **-16** (best **-9**, final **-18**). Slowing exploration slightly (**epsilon_decay = 0.99999**) produced the **best final reward (-9)** and matched the baseline best reward (-9), although its mean performance (**-16.64**) did not improve. Increasing the replay warm-up size to **20,000** or reducing it to **5,000** led to similar mean rewards (around **-17**) with slightly better final rewards (**-15**) than the baseline. More frequent target updates (**target_update = 500**) improved the final reward to **-14** but did

not change overall stability. Increasing the update-to-collection ratio (**train_step = 2 or 5**) degraded performance, giving the lowest mean rewards (**-17.84** and **-19.48**) and the worst final rewards (**-19** and **-20**), suggesting that over-updating the network with limited replay diversity can harm learning in this setting.

Table 3. Metrics of Pong ablation studies

Setting	Mean Eval Reward	Best Eval Reward	Final Eval Reward	Total Env Steps	Total Updates
Baseline	-16	-9	-18	3074267	3064268
Epsilon_deacy_0.99999	-16.64	-9	-9	3041679	3031680
Epsilon_deacy_0.999999	-17.16	-11	-18	3143262	3133263
Replay_5000	-16.96	-12	-15	3098603	3093604
Replay_20000	-17	-11	-15	3193856	3173857
Target_update_500	-16.92	-10	-14	3084399	3074400
Target_update_5000	-16.6	-9	-14	3273148	3263149
Train_setp_2	-17.84	-15	-19	2805172	5570346
Train_setp_5	-19.48	-16	-20	2043616	10118085



(Figure 3. Pong ablations over different setting within episodes and reward.)

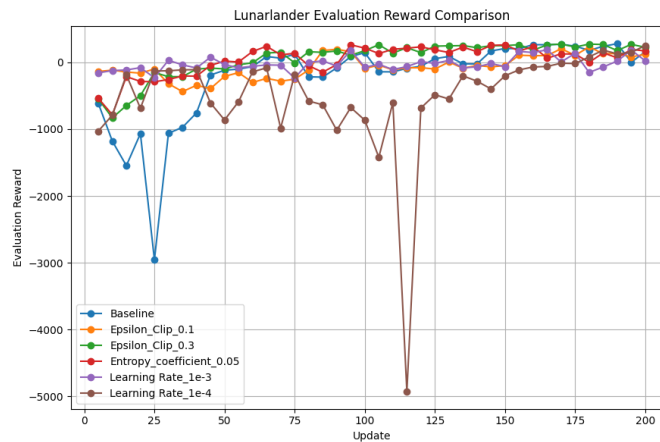
5.2 LunarLander results

As shown in **Table 4** and **Figure 4**, the **LunarLander** environment ablation results show that PPO performance is sensitive to several key hyperparameters, especially the clipping range, entropy regularization, and learning rate. In the baseline setting, the agent achieved a **best evaluation reward of 278.33** and a **final evaluation reward of 158.19**, reaching the predefined threshold at **update 150**. Adjusting the clipping

parameter to $\epsilon = 0.3$ led to the strongest overall performance, improving both the **mean evaluation reward (55.66)** and **final evaluation reward (223.88)**, and reaching the threshold earlier (**update 105**). Increasing the entropy coefficient to **0.3** also reached the threshold quickly (**update 65**) and achieved a solid final reward (**170.37**), indicating that stronger exploration regularization can speed up early learning. In contrast, **entropy coefficient = 0.05** performed poorly, with a low final reward (**22.00**) and failing to reach the threshold. The learning rate study shows that 1×10^{-3} produced a higher final reward (**249.60**) but reached the threshold later (**update 200**) and exhibited more instability in Figure 4, while the baseline learning rate remained more stable overall. Overall, the results suggest that a **larger PPO clipping range ($\epsilon = 0.3$)** provides the most effective and stable improvement among the tested settings, as supported by both the summary metrics and the evaluation reward curves.

Table 4. Metrics of LunarLander ablation studies

Setting	Mean Eval Reward	Best Eval Reward	Final Eval Reward	Threshold Updated
Baseline	-213.092	278.3284	158.1894	150
Epsilon_Clip_0.1	-66.086	230.9373	119.4103	170
Epsilon_Clip_0.3	55.66376	272.8229	223.8816	105
Entropy_coefficient_0.3	34.12694	261.2924	170.373	65
Entropy_coefficient_0.05	-23.2102	187.5816	22.00034	Not reach
Learning Rate_1x10 ⁻³	-491.342	249.5953	249.5953	200
Learning Rate_1x10 ⁻⁴	-213.092	278.3284	158.1894	150



(Figure 4. LunarLander ablations over different setting within episodes and reward.)

6. Discussion

6.1 DQN

DQN agent on Pong showed initial improvement but limited final performance. Under the baseline setting, the evaluation reward improved from around -21 and reached a best value of -9 , but the final evaluation reward returned to -18 , indicating that learning remained noisy and unstable in the Atari visual setting. The ablation results further suggest that Pong is highly sensitive to training dynamics: slight changes in exploration decay and target-network update frequency produced only modest improvements, while increasing the update-to-collection ratio (train steps per environment step) clearly degraded performance. This indicates that, in this implementation, over-updating the network using replay data can harm learning stability, and Atari learning may require stronger stabilization or longer training to achieve consistently high rewards.

6.2 PPO

In contrast to Pong, PPO on LunarLander achieved substantially stronger and more stable learning. The baseline PPO run reached a best evaluation reward of 278.33 and a final evaluation reward of 158.19, and it reached the predefined threshold at update 150, suggesting that PPO can learn an effective policy under the baseline setting. The ablation results show that PPO performance is particularly sensitive to clip range, entropy regularization, and learning rate. For example, using a larger clipping parameter ($\epsilon=0.3$) produced the strongest overall results, improving both mean evaluation reward and final evaluation reward, and reaching the threshold earlier than the baseline. Meanwhile, an inappropriate entropy coefficient (0.05) failed to reach the threshold and led to a low final reward, indicating that exploration regularization needs to be tuned carefully.

6.3 Ablation

Across both tasks, the ablation studies highlight that reinforcement learning performance depends strongly on training design choices, not only on the algorithm itself. For DQN on Pong, changes related to exploration and update scheduling had noticeable effects, and overly aggressive update frequency (higher train steps per data collection) consistently reduced performance, suggesting that replay diversity and update stability are critical in the Atari setting. For PPO on LunarLander, hyperparameters that directly constrain policy updates (clip range) and control exploration (entropy coefficient) showed clear impact on both sample efficiency (earlier threshold update) and final performance. Overall, these results support the message that good RL performance is achieved by matching algorithm behavior and hyperparameter choices to the environment characteristics (visual/high-dimensional Atari vs structured low-dimensional control).

6.4 Limitations

This assignment has several limitations. First, results were reported for single runs per setting, and the experiments did not include multiple random seeds; therefore,

the observed differences may be affected by run-to-run variance, especially in Pong where training is highly noisy. Second, the training budget was limited (e.g., 500 episodes for Pong and 200 updates for PPO), which may not be sufficient to fully optimize Atari performance. Third, the Pong DQN implementation did not include common Atari stabilizing techniques (e.g., reward clipping, frame skipping wrappers, Double DQN, or prioritized replay), which may explain why the best evaluation reward remained far from a solved level. Finally, the threshold definition affects the sample-efficiency metrics; a different threshold could change `threshold_episode`, `threshold_env_step`, or `threshold_update`, so these metrics should be interpreted as relative comparisons under the same evaluation rule rather than absolute performance guarantees.

6.5 Summary

Across both tasks, the results highlight a clear contrast between learning in a high-dimensional Atari environment and a lower-dimensional control environment, as well as between value-based and policy-based training dynamics. For **Pong (DQN)**, the agent showed early improvement (briefly reaching a best evaluation reward around -9) but overall performance remained limited and unstable, and several ablations only produced modest changes; notably, increasing the update-to-data ratio (larger `train_step`) consistently degraded performance, suggesting that over-updating from replay can harm stability in this setting. In comparison, **LunarLander (PPO)** achieved substantially stronger and more reliable learning, reaching high best/final rewards and meeting the predefined threshold within training; the ablation results further indicate that PPO is sensitive to policy-update constraints and exploration regularization, where a larger clipping range (e.g., `clip_eps=0.3`) provided the most effective overall improvement, while poorly tuned entropy coefficients could significantly reduce performance or delay threshold achievement. Overall, these experiments suggest that successful RL performance depends strongly on matching algorithm behavior and hyperparameter choices to the environment characteristics: Atari tasks tend to be noisier and harder to stabilize with basic DQN tuning alone, whereas PPO can learn effective policies more stably in structured control tasks when key hyperparameters (clip range, entropy, and learning rate) are properly tuned.

7.Reference

1. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *nature*, 518(7540), 529-533.
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
3. <https://chatgpt.com/>
4. <https://gymnasium.farama.org/index.html>
5. <https://ale.farama.org/environments/pong/>
6. https://gymnasium.farama.org/environments/box2d/lunar_lander/
7. numpy 2.1.2
8. torch 2.7.1+cu128

8. Appendix: Code Listing

File name	Function
atari_dqn.py	Pong (Atari) training with CNN-based DQN, including Atari preprocessing (grayscale + 84×84 resize + 4-frame stacking), replay buffer, target network, periodic evaluation, and saving training_history.npy / training_summary.npy.
ll_ppo.py	LunarLander-v2 training with PPO (actor–critic), including GAE advantage estimation, PPO clipping objective, entropy regularization, periodic evaluation, and saving training_history.npy / training_summary.npy.
plot.ipynb	Post-training visualization and result aggregation: loads training_history.npy / training_summary.npy across baseline and ablation runs to generate formatted tables and evaluation reward curves for the report.

1. atari_dqn.py

This script implements the **Atari Pong** task using a **CNN-based DQN** agent. It includes an Atari preprocessing pipeline that converts RGB frames to grayscale, resizes them to **84×84**, and stacks **4 consecutive frames** as the input state. The agent is trained with an ϵ -greedy exploration strategy, a replay buffer, and a target network. During training, the script performs periodic evaluation (every 20 episodes), tracks the best evaluation reward, and saves both checkpoints and training logs. The outputs include: (1) training_history.npy for learning curves (train, eval reward vs episodes/steps), (2) training_summary.npy for aggregated metrics (mean/best/final evaluation reward, threshold info, total steps/updates), and (3) model weights (best_model.pt and periodic checkpoints).

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--env-name", type=str, default="ALE/Pong-v5")
    parser.add_argument("--save-dir", type=str, default="/workspace/535505/ass3/save_model/atari_results/bs_4")
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--memory-size", type=int, default=100000)
    parser.add_argument("--lr", type=float, default=1e-4)
    parser.add_argument("--discount-factor", type=float, default=0.99)
    parser.add_argument("--epsilon-start", type=float, default=1.0)
    parser.add_argument("--epsilon-decay", type=float, default=0.99999)
    parser.add_argument("--epsilon-min", type=float, default=0.01)
    parser.add_argument("--target-update-frequency", type=int, default=5000)
    parser.add_argument("--replay-start-size", type=int, default=100000)
    parser.add_argument("--max-episode-steps", type=int, default=10000)
    parser.add_argument("--train-per-step", type=int, default=1)
    parser.add_argument("--episodes", type=int, default=500)
    parser.add_argument("--frame-stack", type=int, default=4)
    parser.add_argument("--reward-threshold", type=float, default=18.0)
    args = parser.parse_args()

    agent = DQNAgent(env_name=args.env_name, args=args)
    agent.run(episodes=args.episodes)
```

2. ll_ppo(1).py

This script implements **PPO** for **LunarLander-v3**. The agent collects rollouts (rollout_steps=2048), computes advantages using **GAE**, and optimizes the policy using the PPO **clipped objective** with entropy regularization and value loss. Evaluation is performed at fixed update intervals (e.g., every 5 updates), and the best checkpoint is saved based on evaluation reward. Similar to the DQN pipeline, the script produces: (1) training_history.npy containing update-wise training statistics (e.g., eval_reward, policy_loss, value_loss, entropy), and (2) training_summary.npy containing aggregated performance metrics (mean, best, final evaluation reward, threshold_update and total environment steps).

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()

    parser.add_argument("--env-name", type=str, default="LunarLander-v3")
    parser.add_argument("--save-dir", type=str, default="/workspace/535505/ass3/save_model/ppo_results/efc805")

    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--gamma", type=float, default=0.99)
    parser.add_argument("--gae-lambda", type=float, default=0.95)
    parser.add_argument("--clip-eps", type=float, default=0.2)
    parser.add_argument("--entropy-coef", type=float, default=0.05)
    parser.add_argument("--value-coef", type=float, default=0.5)
    parser.add_argument("--max-grad-norm", type=float, default=0.5)

    parser.add_argument("--rollout-steps", type=int, default=2048)
    parser.add_argument("--update-epochs", type=int, default=10)
    parser.add_argument("--mini-batch-size", type=int, default=64)
    parser.add_argument("--max-episode-steps", type=int, default=1000)
    parser.add_argument("--total-updates", type=int, default=200)
    parser.add_argument("--eval-every", type=int, default=5)

    parser.add_argument("--reward-threshold", type=float, default=200.0)

    args = parser.parse_args()

    set_seed(42)

    agent = PPOAgent(env_name=args.env_name, args=args)
    agent.run()
```

3. plot.ipynb

This notebook is used for **post-training analysis** and **figure** and **table generation**. By generates evaluation reward curves (single-run and multi-run comparisons), prints the summary metrics in a table-friendly format, and saves the resulting plots for inclusion in the report.

An easier way to run both **DQN** and **PPO** model, we can prompt the file name of model, we can further choose which hyperparameter we would like to test or observe, as shows below.

```
root@70e2366d0cb8:/workspace/535505/ass3# python3 ll_ppo.py --gamma=0.9
root@70e2366d0cb8:/workspace/535505/ass3# python3 atari_dqn.py --lr=25e-4 --discount-factor=0.95
```